

Capacity Planning for Spikes, Worked Out by Hand

PROVISION FOR THE PEAK, PAY FOR THE AVERAGE — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It takes a workload with a $10\times$ peak-to-trough swing — daily cycles, a weekend dip, and a $10\times$ Black-Friday spike — and computes, from first principles, the *four-tier capacity stack* that serves it without either over-paying or dropping traffic. We derive the 40/30/20/10 split, the blended cost ($\approx 0.52\times$ pure on-demand), and why each tier is matched to a traffic *percentile*. Every figure is produced by a small reference script (Appendix A, `m08_t02_capacity_tiers_engine.py`).

The one idea. Provisioning for peak means paying $10\times$ for 1% of the year; provisioning for average means dropping traffic eleven months out of twelve. The right answer is *neither* — it is a *stack* of capacity tiers, each committed at a different time horizon, each priced for the slice of the demand distribution it covers. Cheap, long-horizon capacity sits under the demand floor and runs $\sim$ always; expensive burst capacity fires only at the tail.

CONCEPTS COVERED, IN ORDER

the $10\times$ demand profile ($4\times$ daily $\times 2.5\times$ Q4); the four tiers (committed / reserved / on-demand / spill) and their 40/30/20/10 shares; matching tiers to traffic percentiles (P50/P80/P95/P100); the blended-cost computation and the $0.52\times$ result; why neither extreme (100% committed, 100% on-demand) wins; and the pre-warm insurance for a known spike.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The demand profile — where $10\times$ comes from	2
2	Step 1 — The four-tier capacity stack	2
3	Step 2 — <i>Why</i> match tiers to traffic percentiles	3
4	Step 3 — The blended cost: $\approx 0.52\times$ on-demand	3
5	Step 4 — Why neither extreme wins	4
6	Step 5 — Pre-warming for the known spike	4
7	The argument, at a glance	5
7.1	Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The demand profile — where 10× comes from

Nexus's traffic has three superimposed patterns. We multiply them to get the peak-to-trough ratio the capacity stack must absorb.

Peak demand, step by step

Let the quiet overnight *trough* be 100 capacity units (one unit = one serving GPU-hour of headroom). Then:

$$\begin{aligned}\text{daily peak} &= 4 \times \text{trough} = 400 \quad (\text{daytime is } 4\times \text{ the overnight floor}), \\ \text{Q4 peak} &= 2.5 \times \text{daily peak} = 1000 \quad (\text{Black Friday is } 2.5\times \text{ a normal day}), \\ \text{peak-to-trough} &= \frac{1000}{100} = 10 \times . \quad \checkmark\end{aligned}$$

Intuition

The two multipliers are different *kinds* of variation. The 4× daily swing is *predictable and recurring* — it happens every 24 hours, so you can plan baseload around it. The 2.5× Q4 spike is *rare and known-in-advance* — it happens once a year on a date you have circled. Predictable-recurring demand wants *cheap committed* capacity; rare-but-known demand wants *flexible burst* capacity. One number, 10×, hides two completely different provisioning problems.

2 Step 1 — The four-tier capacity stack

Instead of one capacity number, we build a stack of four tiers, each sized as a fraction of peak and committed at a different horizon (and therefore a different price).

Capacity tier sizing

$$\begin{aligned}\text{peak_demand} &= 4 \times \text{daily} \times 2.5 \times \text{Q4} = 10 \times \text{trough} = 1000, \\ \text{committed_cap} &= 0.40 \times \text{peak_demand} = 400 \quad (3\text{-yr commit, P50}), \\ \text{reserved_cap} &= 0.30 \times \text{peak_demand} = 300 \quad (1\text{-yr reserved, P80}), \\ \text{ondemand_cap} &= 0.20 \times \text{peak_demand} = 200 \quad (\text{hourly, P95}), \\ \text{spill_cap} &= 0.10 \times \text{peak_demand} = 100 \quad (\text{multi-cloud, P100}).\end{aligned}$$

The four fractions sum to 1.00: the stack provisions exactly 100% of peak, so the 10× spike is covered — but only the bottom 40% is bought at the cheapest 3-year rate.

Tier	Frac	Units	Cumul.	Rate (\$/hr)	Horizon / pctl
committed	0.40	400	400	1.125	3-yr commit, P50
reserved	0.30	300	700	1.575	1-yr reserved, P80
on-demand	0.20	200	900	4.500	hourly autoscale, P95
spill	0.10	100	1000	5.175	multi-cloud, P100
total	1.00	1000	= 100% of peak provisioned		

Rates are stated as a discount on the on-demand reference of \$4.50/GPU-hr: committed 0.25×, reserved 0.35×, on-demand 1.00×, spill 1.15× (burst capacity carries a *premium*). Longer commitment buys a deeper discount — you trade flexibility for price.

3 Step 2 — Why match tiers to traffic percentiles

The split is not arbitrary. Each tier covers a band of the demand distribution, and the rule is: **the cheapest capacity must carry the most-utilized band.**

Tier (rate)	Covers	Utilization
committed (P50, cheapest)	demand at/below the median — present ~always	~100%
reserved (P80)	the daily peaks above the floor	high
on-demand (P95)	weekly peaks + half the Q4 ramp	intermittent
spill (P100, premium)	only the Black-Friday tail (Nov–Dec)	~1% of year

The percentile-matching rule

Capacity you use ~always should be the cheapest (deepest commitment); capacity you use ~never should be the most flexible (no commitment), even at a premium rate. Committing the cheap rate to the 40% of demand that is *always* present captures the discount on the units that run the most hours — which is where almost all the GPU-hours, and therefore almost all the dollars, actually are.

Watch out

Match the tier shares to *your* demand profile, not to the 40/30/20/10 template. A flatter workload (an internal business chatbot) shifts toward *committed* — its demand floor is most of its demand. A spikier workload (consumer retail) shifts toward *on-demand* and *spill* — its floor is small and its tail is huge. The mistake at either extreme is the same error in opposite directions: mispricing the band you live in.

4 Step 3 — The blended cost: $\approx 0.52\times$ on-demand

Provision the same 100%-of-peak capacity two ways and compare. (a) Everything at the on-demand rate. (b) The tiered blend — each tier’s units at its own discounted rate. The blended \$/GPU-hr is the *capacity-weighted average* of the four tier rates.

Blended rate, step by step

$$\begin{aligned}
 \text{pure on-demand} &= 1000 \times \$4.50 = \$4500/\text{hr}, \\
 \text{tiered blend} &= 0.40(1.125) + 0.30(1.575) + 0.20(4.500) + 0.10(5.175) \\
 &= 0.450 + 0.4725 + 0.900 + 0.5175 = \$2.340/\text{GPU-hr}, \\
 \text{tiered total} &= 1000 \times \$2.340 = \$2340/\text{hr}, \\
 \frac{\text{blended}}{\text{on-demand}} &= \frac{2.340}{4.500} = \mathbf{0.520}. \quad \checkmark
 \end{aligned}$$

The blended cost is 0.52× pure on-demand — a 48% saving on the same provisioned capacity. (The headline “~60% savings” includes the further effect of paying the *burst* tiers only for the hours they are actually switched on; see Step 4.)

Mini-example — the weighting in isolation

A weighted average pulls toward the heaviest weights. Here the two cheapest tiers carry 70% of the capacity ($0.40 + 0.30$) at rates well below on-demand, so they dominate the blend and drag the average down to $0.52\times$. If you had committed only 10% and left 90% on-demand, the blend would barely move off $1.0\times$ — the discount lives in the *weight*, not just the rate.

5 Step 4 — Why neither extreme wins

Two tempting extremes both lose. We compare *annual* spend, charging the burst tiers only for the fraction of the year they actually run (their duty cycle: committed 1.00, reserved 0.85, on-demand 0.30, spill 0.04).

Strategy	Annual spend	Verdict
A. commit 100% of peak, 24/7	\$9.86M	cheap on paper, but locks 3-yr to peak you can't downsize
B. 100% on-demand	\$28.34M	flexible, but full rate everywhere
C. tiered 40/30/20/10	\$10.01M	flexible <i>and</i> near-A cost

The two losing extremes

A — commit everything. You get the deep \$1.125/hr rate, but only by buying all 1000 units every hour of the year, locked for three years. About 60% of that capacity is idle outside Q4, and providers will not grant commit pricing on burst capacity you touch for 1% of the year. The \$9.86M is a *paper* minimum that ignores lock-in risk and the inability to downsize.

B — pure on-demand. Total flexibility, but every served unit pays the full \$4.50/hr; you leave the entire commitment discount on the table. Result: \$28.34M — $2.8\times$ the tiered stack.

The tiered stack is the risk-adjusted optimum

Tiered 40/30/20/10 costs \$10.01M/yr — 65% below pure on-demand, and within 2% of the commit-everything paper minimum, *while keeping the flexibility to scale down and the ability to absorb a $10\times$ spike*. You capture almost all of the commitment discount on the baseload, and you keep optionality on the tail. That is the whole point of the stack.

6 Step 5 — Pre-warming for the known spike

Autoscaling cannot answer a 10-second spike: a cold GPU pod takes 4–9 minutes to warm (pull image, load weights, initialize the KV pool, warm CUDA graphs). For *known* events — Black Friday, a launch, a regulatory deadline — you provision *ahead* of the curve.

The insurance is cheap

annual tiered spend $\approx$ \$2340/hr-equiv blended $\times$ duty-weighted hours $\approx$ \$10.0M,
pre-warm runbook cost $\approx 0.3\% \times \$10.0\text{M} = \$30,000$. ✓

Intuition

The pre-warm runbook (scale to 80% of forecast at T−6h, activate the spillover pool at T−2h, let the autoscaler handle residuals on already-warm pods, scale down at T+18h) costs about 0.3% of annual spend. The alternative — getting caught flat on the biggest traffic day of the year — is hours of elevated error rate, SLO-credit requests, and lost trust. Spending 0.3% to insure the other 99.7% against the one spike you *knew* was coming is not a hard call.

Watch out

Pre-warming only works for spikes you can *see coming*. For unannounced surges (a merchant migration, a viral event) the stack's on-demand and spill tiers are your defense — which is exactly why you keep 30% of peak in flexible capacity rather than committing all of it. The committed tier is for demand you are *certain* about; the flexible tiers are the price of being wrong about the rest.

7 The argument, at a glance

#	Step	Result
0	Demand	$4 \times \text{daily} \times 2.5 \times \text{Q4} = 10 \times \text{peak-to-trough}$
1	Split	40/30/20/10 of peak = committed/reserved/on-demand/spill
2	Percentile	cheapest tier (committed) carries P50 floor, ~100% utilized
3	Blend	$0.40(1.125) + 0.30(1.575) + 0.20(4.5) + 0.10(5.175) = \$2.34 = 0.52 \times \text{OD}$
4	Extremes	tiered \$10.0M vs OD \$28.3M (−65%) vs commit-all \$9.9M (locked)
5	Pre-warm	~0.3% of spend $\approx$ \$30k insures the known spike

7.1 Equation cheat-sheet

Peak demand	$\text{peak} = 4 \times \text{daily} \times 2.5 \times \text{Q4} = 10 \times \text{trough}$
Tier sizing	$\{0.40, 0.30, 0.20, 0.10\} \times \text{peak} \Rightarrow \text{committed/reserved/OD/spill}$
Blended rate	$\bar{r} = \sum_t \text{frac}_t \cdot \text{rate}_t$
Blend ratio	$\bar{r}/\text{rate}_{\text{OD}} \approx 0.52$ ($\approx 48\%$ saving)
Pre-warm cost	$\approx 0.003 \times \text{annual spend}$

Appendix A — Reference implementation

Every number above is produced by `m08_t02_capacity_tiers_engine.py` (provided alongside this PDF). Run it to reproduce the split, the blend, and the extremes; change the demand multipliers or tier discounts to model a different workload (flatter $\Rightarrow$ more committed; spikier $\Rightarrow$ more on-demand).

```

1 """
2 Reference implementation for: Module 08 - Topic 2
3 "Capacity Planning for Spikes - Provision for the peak, pay for the average."
4
5 Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6 so the arithmetic in the document is exact and self-consistent.
7
8 The model: a workload with a 10x peak-to-trough ratio is served by a FOUR-TIER
9 capacity stack, each tier committed at a different time horizon and price:

```

```

10
11     peak_demand    = 4x daily_peak * 2.5x Q4_multiplier = 10x trough
12     committed_cap  = 0.40 * peak_demand    (3-year commit, deepest discount, P50)
13     reserved_cap   = 0.30 * peak_demand    (1-year reserved, mid discount, P80)
14     ondemand_cap   = 0.20 * peak_demand    (hourly, autoscaling)
15     spill_cap      = 0.10 * peak_demand    (multi-cloud, Nov-Dec only)
16     blended_cost   ~ 0.52 * on_demand_cost (~60% savings vs pure on-demand)
17
18 Capacity is in GPU-equivalents (one "unit" = one serving GPU-hour of headroom).
19 Prices are $/GPU-hr at each commitment horizon.
20 ""
21
22 # -----
23 # 0. The demand profile -- where the 10x peak-to-trough comes from.
24 # -----
25 trough            = 100.0                # baseline trough demand (GPU-equiv units)
26 daily_peak_mult   = 4.0                  # daily pattern: peak is 4x the trough
27 q4_mult           = 2.5                  # annual pattern: Black Friday is 2.5x daily peak
28 peak_demand       = trough * daily_peak_mult * q4_mult
29 print("== 0. Demand profile ==")
30 print(f"trough                = {trough:.0f} units")
31 print(f"daily peak (x4)       = {trough*daily_peak_mult:.0f} units")
32 print(f"Q4 peak (x2.5 more)   = {peak_demand:.0f} units")
33 print(f"peak-to-trough ratio = {peak_demand/trough:.0f}x")
34
35 # -----
36 # 1. The four-tier split: 40 / 30 / 20 / 10 of peak.
37 # -----
38 on_demand_price = 4.50                # the "pay-as-you-go" reference rate ($/GPU-hr)
39 # Each tier prices the SAME GPU-hour of capacity differently, set by its
40 # commitment horizon. Longer commitment -> deeper discount (you trade
41 # flexibility for price). Spill is a premium burst rate (above on-demand).
42 # discount is the multiplier on the on-demand rate.
43 tiers = [
44     # name, fraction-of-peak, horizon, discount(x OD rate), percentile
45     ("committed", 0.40, "3-year commit", 0.25, "P50"),
46     ("reserved", 0.30, "1-year reserved", 0.35, "P80"),
47     ("ondemand", 0.20, "hourly autoscale", 1.00, "P95"),
48     ("spill", 0.10, "multi-cloud burst", 1.15, "P100"),
49 ]
50
51 print("\n== 1. Four-tier capacity split (40/30/20/10 of peak) ==")
52 print(f"{'tier':10s}{'frac':>6s}{'units':>8s}{'cumul':>8s}{'disc':>7s}{'$/GPU-hr':>10s}{'pctile':>8s}")
53 cap = {}
54 price = {}
55 cumul = 0.0
56 for name, frac, horizon, disc, pct in tiers:
57     units = frac * peak_demand
58     cumul += units
59     cap[name] = units
60     price[name] = disc * on_demand_price # effective hourly rate for this tier
61     print(f"{'name':10s}{'frac':>6.2f}{'units':>8.0f}{'cumul':>8.0f}{'disc':>7.2f}{'price[name]':>10.3f}{'pct':>8s}")
62 total_cap = cumul
63 print(f"total provisioned capacity = {total_cap:.0f} units = {total_cap/peak_demand:.0%} of peak")
64 print(f"(sum of fractions = {sum(f for _,f,_,_,_ in tiers):.2f})")
65
66 # -----
67 # 2. Why match tiers to traffic percentiles.
68 #     Committed serves the demand floor present ~all the time (P50);
69 #     reserved covers up to P80; on-demand the weekly peaks (P95); spill the
70 #     rare Q4 tail (P100). The cheapest tier carries the most-utilized units.
71 # -----
72 print("\n== 2. Percentile -> tier mapping (cheapest capacity = most used) ==")

```

```

73 print(" committed (P50) : demand at/below this exists ~always -> 100% utilized")
74 print(" reserved (P80) : covers the daily peaks -> high utilized")
75 print(" ondemand (P95) : covers weekly peaks + half the Q4 spike")
76 print(" spill (P100) : only the Black-Friday tail (Nov-Dec) -> ~1% of year")
77
78 # -----
79 # 3. The blended cost vs pure on-demand.
80 # Apples-to-apples: provision the SAME 100%-of-peak capacity two ways.
81 # (a) everything at the on-demand rate;
82 # (b) the tiered blend -- each tier's units at its own discounted rate.
83 # The blended $/GPU-hr is the capacity-weighted average of the tier rates.
84 # -----
85 print("\n== 3. Blended cost vs pure on-demand ==")
86 # (a) provision 100% of peak entirely on-demand
87 all_on_demand = total_cap * on_demand_price
88 # (b) tiered: each tier's units at its own rate
89 tiered_cost = sum(cap[name] * price[name] for name,_,_,_ in tiers)
90 blended_price = tiered_cost / total_cap # capacity-weighted $/hr
91 blend_ratio = blended_price / on_demand_price
92 savings = 1.0 - blend_ratio
93 print("weighted blend = 0.40*{:.3f} + 0.30*{:.3f} + 0.20*{:.3f} + 0.10*{:.3f}".format(
94     price['committed'], price['reserved'], price['ondemand'], price['spill']))
95 print(f"pure on-demand (100% of peak @ ${on_demand_price:.2f}) = ${all_on_demand:,.0f}/hr")
96 print(f"tiered blend (each tier @ its rate) = ${tiered_cost:,.0f}/hr")
97 print(f"blended $/GPU-hr = ${blended_price:.3f}")
98 print(f"blended / on-demand = {blend_ratio:.3f} (~0.52 target)")
99 print(f"savings vs pure on-demand = {savings:.1%} (~48% here; ~60% headline)")
100
101 # -----
102 # 4. Why neither extreme is right -- compare TOTAL ANNUAL SPEND to meet SLA.
103 # The key realism: the tiered stack pays the BURST tiers (on-demand, spill)
104 # only for the HOURS they are actually switched on, while the committed and
105 # reserved baseload run 24/7/365. The two extremes lose because they pay a
106 # flat rate for all 1000 peak units every hour of the year.
107 # duty = fraction of the year each tier is actually carrying load.
108 # -----
109 print("\n== 4. Why neither extreme is right (annual spend to meet SLA) ==")
110 HRS = 24 * 365
111 duty = {"committed":1.00, "reserved":0.85, "ondemand":0.30, "spill":0.04}
112 # A) commit 100% of PEAK at the committed rate, but pay for it 24/7/365:
113 # all 1000 units always on, even though ~60% are idle outside the Q4 tail.
114 commit_all_annual = peak_demand * (tiers[0][3]*on_demand_price) * HRS
115 # B) 100% on-demand at full rate, paid only for active hours per tier-share.
116 ondemand_annual = sum(cap[n]*on_demand_price*duty[n]*HRS for n,_,_,_ in tiers)
117 # C) tiered: each tier at its own rate, paid only for its active hours.
118 tiered_annual = sum(cap[n]*price[n]*duty[n]*HRS for n,_,_,_ in tiers)
119 print(f"A) commit 100% of peak, 24/7 : ${commit_all_annual:,.0f}/yr (cheap on paper, but locks 3-yr to peak y
120 print(f" can't downsize; providers won't commit-rate burst capacity)")
121 print(f"B) 100% on-demand : ${ondemand_annual:,.0f}/yr (flexible, but full rate -- no discount)")
122 print(f"C) tiered 40/30/20/10 : ${tiered_annual:,.0f}/yr <-- flexible AND near commit-all cost")
123 print(f" tiered vs on-demand : {tiered_annual/ondemand_annual:.2f}x ({(1-tiered_annual/ondemand_annual)
124 print(f" tiered vs commit-all : {tiered_annual/commit_all_annual:.2f}x (within {abs(1-tiered_annual/com
125
126 # -----
127 # 5. Pre-warm insurance cost for the known Q4 spike.
128 # -----
129 print("\n== 5. Pre-warm insurance (known spike) ==")
130 prewarm_cost = 0.003 * tiered_annual # ~0.3% of annual tiered spend
131 print(f"annual tiered spend ~ ${tiered_annual:,.0f}")
132 print(f"pre-warm runbook cost ~0.3% = ${prewarm_cost:,.0f} (cheap insurance vs a flat-footed outage)")

```

— END OF DOCUMENT —